MINI TRACK MONITORING SYSTEM

(MTMS)

Complete Software Requirement Document

Specification, Architecture, Delivery Guide and Reference Implementation

Version 1.0

Technology: Core Java JDK 21, JDBC and PostgreSQL

Project Type: Simple IntelliJ Java Project

Backend-only miniature project aligned to the future MTDL-SAD backend

Document Control

Item	Details
Document title	Mini Track Monitoring System (MTMS) – Complete Requirement Document
Version	1.0
Document type	Software Requirement Specification, delivery guide and reference solution
Primary technology	Java JDK 21, JDBC, PostgreSQL
Development environment	IntelliJ IDEA – simple Java project, no Maven or Gradle
Prepared for	Trainer (in role of a Team-Lead) led developer workshop
Expected duration	Minimum 6 hours
Application type	Console-based backend application

How to Use This Document

1. Developers read the functional requirements and build each module in the proposed order.
2. Developers use the acceptance criteria and test cases to check their own work.
3. The trainer uses the delivery notes and checkpoints to reduce repeated intervention.
4. The reference implementation in the appendices is shown only after developers complete their attempt.

Important Scope Rule
This is a backend-only project and at present we are not adding a graphical interface, web interface, REST API, Spring Boot, Maven, Gradle, JSON processing, UDP communication, Redis, Scanner input or any front-end technology.

Table of Contents

1. Executive Summary
2. Project Background and Business Problem
3. Project Scope
4. Technology and Constraints
5. Business and Functional Requirements
6. Non-Functional Requirements
7. Detailed Architecture
8. Module Specifications
9. Data Model and Database Design
10. Project Structure and Setup
11. Development Plan

12. Testing and Acceptance

13. Requirements Traceability Matrix

14. Developer Deliverables

15. Trainer Delivery Notes

16. Connection to MTDL-SAD

Appendix A. SQL Scripts

Appendix B. Complete Java Reference Code

Appendix C. Expected Output and Viva Questions

1. Executive Summary

The Mini Track Monitoring System (MTMS) is a console-based backend application for receiving simulated information about moving objects. A moving object is represented by a Track. A Track can represent an aircraft, ship, drone or vehicle. The system validates each Track, stores valid Tracks in PostgreSQL, stores validation problems as alerts, shows simple reports and replays saved Tracks in time order.

The project is deliberately small enough to complete in a day's session detailed enough to scale up to the actual MTDL software that we will be building further as it follows the same basic engineering ideas used in larger backend systems: domain objects, validation, services, repositories, JDBC, database tables, reporting, error handling and replay.

Developers will build the system using only Core Java JDK 21 and JDBC linked to PostgreSQL RDBMS. They will not use Spring Boot, Maven, REST APIs, JSON, UDP, Redis or a front end. This removes unnecessary difficulty while preserving the backend flow that will later appear in the larger MTDL-SAD project.

Simple Project Story:

A simulator creates Track objects. The system checks every Track. Valid Tracks are saved. Invalid Tracks create Alerts. Saved Tracks can be searched, reported and replayed.

2. Project Background and Business Problem

2.1 What Is a Track?

A Track is a record that describes the position and movement of an object at a particular time. In this miniature project, each Track contains a track number, latitude, longitude, speed, identity, status and created time.

Field	Simple meaning	Example
Track number	Unique business label for the moving object	TRK1001
Latitude	North or south position	19.076000
Longitude	East or west position	72.877700
Speed	Current movement speed used by this simulation	650
Identity	Classification of the Track	FRIEND
Status	Whether the Track is active or inactive	ACTIVE
Created time	Time when the Track object was created	Current date and time

2.2 Current Anticipated Problems

a. Incorrect latitude or longitude may be stored.
b. A negative or unrealistic speed may be accepted.
c. An unknown identity value may enter the database.
d. Invalid data may not be visible to the operator.
e. Historical Tracks may exist but cannot be replayed in sequence.
f. Reports such as Hostile Tracks or high-speed Tracks may require manual checking.

g. Database code and business logic may become mixed and difficult to maintain.

2.3 Proposed Solution

MTMS separates the work into clear layers. The Simulator creates Track objects. The Service layer applies rules. The Repository layer performs SQL operations. PostgreSQL stores Track and Alert records. Replay and analytics services read stored information and present it in the console.

3. Project Scope

3.1 In Scope

a. Create Track objects automatically without user input.
b. Validate every Track before saving it.
c. Return all validation errors, not only the first error.
d. Save valid Tracks into PostgreSQL with JDBC.
e. Create and save Alerts for every validation failure.
f. Retrieve all Tracks.
g. Find a Track by track number.
h. Delete a Track by track number for demonstration.
i. Show Hostile Track and high-speed Track reports.
j. Show identity-wise Track counts.
k. Replay Tracks in created-time order with a one-second delay.
l. Use a clean package-based layered structure.

3.2 Out of Scope

but will be added later on to complete the actual MTDL

a. No front end or browser screen.
b. No Scanner or keyboard input.
c. No Spring Boot or other framework.
d. No Maven or Gradle project.
e. No REST API.
f. No JSON parsing.
g. No UDP networking.
h. No Redis.
i. No Docker.
j. No military symbol rendering.
k. No real tactical data. All data is synthetic and used only for software development practice.

3.3 Environment Setup

We request the developers to conduct all these preliminary checks of Tech-stack before coding:

a. Java JDK 21 is installed.
b. PostgreSQL is installed and running.
c. A local database user is available.
d. The PostgreSQL JDBC driver JAR is added manually to IntelliJ.
e. Database connection values are updated in DBConnection class before execution.

4. Technology and Development Constraints

Area	Required choice	Notes
Language	Java JDK 21	Use standard language features only.
Database	PostgreSQL	Local installation is sufficient.
Database API	JDBC	Use Connection, PreparedStatement and ResultSet.
IDE	IntelliJ IDEA	Use a simple Java project.
Dependency handling	Manual JAR	Add PostgreSQL JDBC driver under lib.
Input	Predefined objects	Do not use Scanner.
Output	Console	Use clear headings and messages.
Build framework	None	No Maven or Gradle.
Backend framework	None	No Spring Boot.

5. Business and Functional Requirements

ID	Business requirement	Explanation
BR-01	Reliable Track processing	The system must process simulated Track data in a predictable manner.
BR-02	Data quality	Invalid data must not be stored in the Tracks table.
BR-03	Traceability	Every validation problem must create an Alert record.
BR-04	Historical review	Stored Tracks must be replayed in sequence.
BR-05	Operational awareness	The system must provide Hostile, high-speed and identity-count reports.

ID	Requirement	Detailed rule	Expected result
FR-01	Generate Tracks	TrackSimulator shall return a List<Track> containing valid and invalid examples. No keyboard input is allowed.	A list of Track objects is created.
FR-02	Validate track number	Track number shall not be null, blank or shorter than five characters.	Errors are returned when any rule fails.
FR-03	Validate latitude	Latitude shall be from -90 to +90 inclusive.	Out-of-range latitude is rejected.

ID	Requirement	Detailed rule	Expected result
FR-04	Validate longitude	Longitude shall be from -180 to +180 inclusive.	Out-of-range longitude is rejected.
FR-05	Validate speed	Speed shall be greater than 0 and not greater than 2000.	Invalid speed is rejected.
FR-06	Validate identity	Identity shall be FRIEND, HOSTILE, NEUTRAL or UNKNOWN.	Other values are rejected.
FR-07	Validate status	Status shall be ACTIVE or INACTIVE.	Other values are rejected.
FR-08	Save valid Track	TrackService shall call TrackRepository only if there are no validation errors.	A row is inserted into tracks.
FR-09	Generate Alerts	One Alert shall be created for each validation error.	Invalid Track data does not enter tracks.
FR-10	Save Alerts	AlertRepository shall insert the Alert into alerts.	Alert history remains available.
FR-11	Display all Tracks	TrackService shall retrieve and print all Tracks.	All stored rows are visible.
FR-12	Search Track	The system shall find a Track using track number.	Matching details or Not Found is shown.
FR-13	Hostile report	The system shall display Tracks whose identity is HOSTILE.	Only Hostile Tracks appear.
FR-14	High-speed report	The system shall display Tracks whose speed is greater than 700.	Only matching Tracks appear.
FR-15	Statistics	The system shall count FRIEND, HOSTILE, NEUTRAL and UNKNOWN Tracks.	Four counts are displayed.
FR-16	Replay history	ReplayService shall read Tracks ordered by created_time and track_id, display each Track and wait one second.	Replay begins, processes all saved Tracks and completes.
FR-17	Delete Track	TrackRepository shall delete a Track using track number.	Affected row count is returned.
FR-18	Handle failures	The main class shall catch unexpected exceptions and print a clear error.	Application does not fail silently.

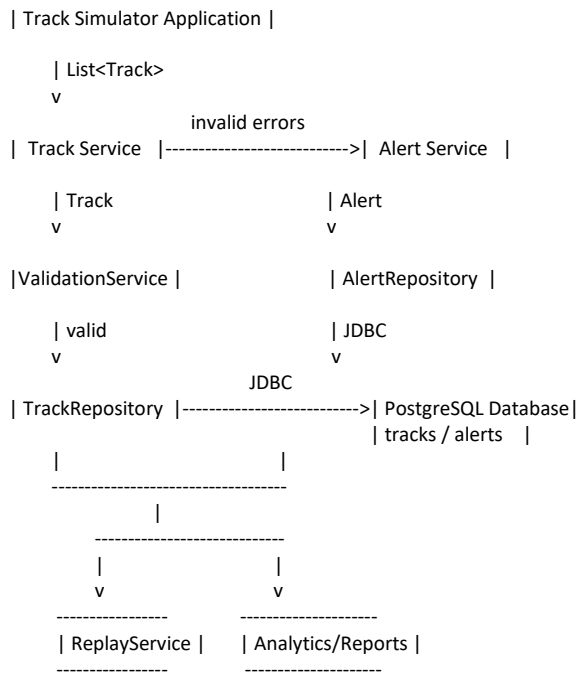
6. Non-Functional Requirements

ID	Quality area	Requirement
NFR-01	Simplicity	Code shall use beginner-friendly class and method names.

ID	Quality area	Requirement
NFR-02	Maintainability	Business logic and database logic must be in separate packages.
NFR-03	Reliability	Try-with-resources must close JDBC resources.
NFR-04	Security basics	All dynamic SQL values must use PreparedStatement parameters.
NFR-05	Performance	The project should process at least 100 simulated Tracks in one execution on a normal development machine.
NFR-06	Readability	Console messages shall clearly state success, rejection, reports and replay status.
NFR-07	Portability	Only connection settings should need change for another local PostgreSQL environment.
NFR-08	Extensibility	The project structure shall allow the simulator or repository to be replaced later.
NFR-09	No hidden input	The application must run without pausing for Scanner input.
NFR-10	Data consistency	Database CHECK constraints shall repeat important validation rules.

7. Detailed Architecture

7.1 High-Level Architecture



7.2 Layer Responsibilities

Layer	Classes	Responsibility	Must not do
Main	MTMSApplication	Creates services and controls the demonstration flow.	Must not contain SQL or validation rules.
Domain	Track, Alert	Stores business data in objects.	Must not connect to the database.
Simulator	TrackSimulator	Creates predefined Track objects.	Must not insert directly into PostgreSQL.
Service	ValidationService, TrackService, AlertService, ReplayService, TrackAnalyticsService	Applies business rules and coordinates repositories.	Must not contain connection URLs.
Repository	TrackRepository, AlertRepository	Runs SQL and maps ResultSet rows into objects.	Must not decide whether data is valid.
Utility	DBConnectionUtil	Creates JDBC connections.	Must not contain business processing.
Constants	MTMSConstants	Keeps repeated fixed values.	Must not perform logic.

7.3 Processing Sequence

a. MTMSApplication asks TrackSimulator to generate sample Tracks.
b. TrackSimulator returns a List of Track objects.
c. MTMSApplication sends each Track to TrackService.processTrack().
d. TrackService asks ValidationService for all validation errors.
e. If the error list is empty, TrackService calls TrackRepository.saveTrack().
f. If errors exist, TrackService calls AlertService once for every error.
g. AlertService creates an Alert and calls AlertRepository.saveAlert().
h. Reports read saved data through repositories.
i. ReplayService requests replay data ordered by created time and displays one record per second.

7.4 Track Lifecycle

```
CREATED -> VALIDATED -> STORED -> REPORTED -> REPLAYED -> REJECTED -> ALERT CREATED -> ALERT STORED
```

7.5 JDBC Flow

```
Java Service
  -> Repository method
    -> DBConnectionUtil.getConnection()
      -> PreparedStatement with ? parameters
        -> executeUpdate() or executeQuery()
          -> ResultSet mapping
            -> Domain object / affected row count
              -> Automatic resource closing
```

8. Module Specifications

Class	Layer	Detailed responsibility
Track	Domain	Holds trackNumber, latitude, longitude, speed, identity, trackStatus and createdTime. Provides constructor, getters and toString.
Alert	Domain	Holds trackNumber, alertMessage, alertSeverity and alertTime. Provides constructor, getters and toString.
TrackSimulator	Simulator	Builds a mixed list of valid and invalid Track objects. Keeps input deterministic and removes Scanner dependency.
ValidationService	Service	Checks every field and returns List<String>. It must not stop after the first problem.
TrackService	Service	Central coordinator. Validates, saves, creates Alerts, displays Tracks, searches, reports statistics and removes Tracks.
AlertService	Service	Chooses severity, creates Alert and sends it to AlertRepository.
ReplayService	Service	Reads replay records through TrackRepository and displays them in order with a configurable delay.
TrackAnalyticsService	Service	Shows total, Hostile and high-speed Track reports using repository queries.

Class	Layer	Detailed responsibility
TrackRepository	Repository	Implements insert, select all, search, count, filtered queries, delete and replay query.
AlertRepository	Repository	Implements insert and select all Alerts.
DBConnectionUtil	Utility	Contains database URL, user and password and returns DriverManager connection.
MTMSApplication	Main	Runs the complete demonstration without user input.

9. Data Model and Database Design

9.1 Relationship Explanation

Tracks and Alerts are not connected with a database foreign key in this miniature project because an invalid Track is deliberately not stored in the Tracks table. The Alert still keeps the attempted track number as text, allowing the problem to be traced without requiring a valid Track row.

9.2 Tracks Data Dictionary

Column	Type	Required	Rule	Purpose
track_id	BIGSERIAL	Yes	Primary key	Database identity and replay tie-breaker
track_number	VARCHAR(20)	Yes	Not blank	Business Track identifier
latitude	NUMERIC(10,6)	Yes	-90 to 90	Current north/south position
longitude	NUMERIC(10,6)	Yes	-180 to 180	Current east/west position
speed	NUMERIC(10,2)	Yes	>0 and <=2000	Movement speed for the simulation
identity_type	VARCHAR(20)	Yes	Allowed identity list	Track classification
track_status	VARCHAR(20)	Yes	ACTIVE or INACTIVE	Current business status
created_time	TIMESTAMP	Yes	Not null	Creation time used by replay

9.3 Alerts Data Dictionary

Column	Type	Required	Rule	Purpose
alert_id	BIGSERIAL	Yes	Primary key	Database identity
track_number	VARCHAR(20)	No	May be unknown	Attempted Track identifier
alert_message	VARCHAR(500)	Yes	Not blank	Validation failure explanation

Column	Type	Required	Rule	Purpose
alert_severity	VARCHAR(20)	Yes	LOW, MEDIUM or HIGH	Importance level
alert_time	TIMESTAMP	Yes	Not null	Time alert was generated

9.4 Key SQL Queries

All Tracks
SELECT * FROM tracks ORDER BY created_time, track_id;

Search by number
SELECT * FROM tracks WHERE track_number = ? ORDER BY created_time DESC, track_id DESC LIMIT 1;

Hostile Tracks
SELECT * FROM tracks WHERE identity_type = 'HOSTILE' ORDER BY created_time;

High-speed Tracks
SELECT * FROM tracks WHERE speed > ? ORDER BY speed DESC;

Replay
SELECT * FROM tracks ORDER BY created_time, track_id;

Identity statistics
SELECT identity_type, COUNT(*) FROM tracks GROUP BY identity_type;

10. Project Structure and Setup

```
MTMS/
 ├── lib/
|   └── postgresql-42.x.x.jar
 ├── database/
|    ├── 01_create_database.sql
|    ├── 02_create_tables.sql
|    ├── 03_sample_data.sql
|   └── 04_verification_queries.sql
└── src/
    └── com/
        └── mtms/
            ├── main/
            |   └── MTMSApplication.java
            ├── domain/
            |    ├── Track.java
            |   └── Alert.java
            ├── simulator/
            |   └── TrackSimulator.java
```

```
├── service/
|   ├── ValidationService.java
|   ├── TrackService.java
|   ├── AlertService.java
|   ├── ReplayService.java
|   └── TrackAnalyticsService.java
├── repository/
|   ├── TrackRepository.java
|   └── AlertRepository.java
├── util/
|   └── DbConnection.java
└── constants/
    └── MTMSConstants.java
```

10.1 IntelliJ Setup

a. Create a new Java project and select JDK 21.
b. Do not select Maven or Gradle.
c. Create the package structure exactly as shown.
d. Create a lib folder in the project root.
e. Place the PostgreSQL JDBC driver JAR in lib.
f. Open Project Structure, Modules, Dependencies and add the JAR.
g. Create the mtmsdb database and run the table script.
h. Update URL, user and password in DBConnectionUtil.
i. Run MTMSApplication.

11. Development Plan

Possible Time	Activity	Expected checkpoint
00:00–00:30	Requirement and architecture walkthrough	Developers can understand Track, Alert, validation, repository and replay.
00:30–01:00	Database and project setup	Database created, tables created, driver configured.
01:00–01:45	Domain classes and constants	Track and Alert compile successfully.
01:45–02:30	Validation and simulator	Valid and invalid Track objects tested in memory.
02:30–03:45	JDBC utility and repositories	Insert and select operations work.
03:45–04:45	Services and processing flow	Valid Tracks saved and invalid Tracks create Alerts.
04:45–05:30	Reports and replay	Hostile, high-speed, statistics and replay work.
05:30–06:00	Testing, bug fixing and demonstration	Acceptance checklist completed.
After 06:00	Stretch work	Improve queries, severity, reports or processing volume.

12. Testing and Acceptance

12.1 Test Cases – check-points

ID	Scenario	Input / setup	Expected result
TC-01	Valid Track	TRK1001, valid coordinates, 650, FRIEND, ACTIVE	Track inserted; no Alert.
TC-02	Blank Track number	Blank string	Track rejected; Track Number Cannot Be Empty Alert.
TC-03	Short Track number	TRK	Track rejected; Track Number Too Short Alert.
TC-04	Invalid latitude	95	Track rejected; latitude Alert.
TC-05	Invalid longitude	200	Track rejected; longitude Alert.
TC-06	Zero speed	0	Track rejected; speed Alert.
TC-07	Excess speed	2500	Track rejected; allowed-limit Alert.
TC-08	Invalid identity	ENEMY	Track rejected; identity Alert.
TC-09	Invalid status	RUNNING	Track rejected; status Alert.
TC-10	Multiple failures	Bad latitude, speed and identity	One Alert saved for every failure.
TC-11	Search existing	TRK1001	Track details displayed.
TC-12	Search missing	TRK9999	Track Not Found displayed.
TC-13	Hostile report	Saved mixed data	Only HOSTILE Tracks displayed.
TC-14	High-speed report	Speeds above and below 700	Only speeds >700 displayed.
TC-15	Replay	Three stored Tracks	All displayed in created-time order with delay.
TC-16	Database unavailable	Stop PostgreSQL	Clear application error printed.
TC-17	SQL injection-like number	TRK1' OR '1'='1	PreparedStatement treats value as data.
TC-18	Empty database	No Tracks	Reports show empty result and replay explains no data.

12.2 Acceptance Criteria once pushed to GitHub

a. Project opens and compiles in IntelliJ with JDK 21.
b. No Maven, Gradle, Spring Boot, Scanner, JSON, UDP, Redis or front end is present.
c. Valid Tracks are inserted into tracks.

d. Invalid Tracks are not inserted into tracks.
e. Every validation failure produces an Alert in alerts.
f. All JDBC statements use PreparedStatement.
g. JDBC resources use try-with-resources.
h. All Tracks, search, reports and statistics work.
i. Replay order is deterministic using created_time and track_id.
j. The main class runs from beginning to end without user input.
k. Developer can explain the responsibility of every package.

13. Requirements Traceability Matrix

Req.	Requirement	Implementation	DB	Test	Status
FR-01	Generate Tracks	TrackSimulator	None	TC-01	Planned
FR-02	Validate track number	ValidationService	None	TC-02, TC-03	Planned
FR-03	Validate latitude	ValidationService	None	TC-04	Planned
FR-04	Validate longitude	ValidationService	None	TC-05	Planned
FR-05	Validate speed	ValidationService	None	TC-06, TC-07	Planned
FR-06	Validate identity	ValidationService	None	TC-08	Planned
FR-07	Validate status	ValidationService	None	TC-09	Planned
FR-08	Save valid Track	TrackService, TrackRepository	tracks	TC-01	Planned
FR-09	Generate Alerts	TrackService, AlertService	alerts	TC-10	Planned
FR-10	Save Alerts	AlertRepository	alerts	TC-10	Planned
FR-11	Display all Tracks	TrackService, TrackRepository	tracks	TC-18	Planned
FR-12	Search Track	TrackService, TrackRepository	tracks	TC-11, TC-12	Planned
FR-13	Hostile report	TrackService or TrackAnalyticsService	tracks	TC-13	Planned
FR-14	High-speed report	TrackService or TrackAnalyticsService	tracks	TC-14	Planned
FR-15	Statistics	TrackService	tracks	TC-01	Planned
FR-16	Replay history	ReplayService, TrackRepository	tracks	TC-15	Planned
FR-17	Delete Track	TrackService, TrackRepository	tracks	Manual verification	Planned
FR-18	Handle failures	MTMSApplication	Both	TC-16	Planned

14. Developer Deliverables

a. Complete IntelliJ simple Java project.
b. PostgreSQL database scripts.
c. All Java classes in the required packages.
d. Screenshot or console capture showing valid Track storage.
e. Screenshot or SQL result showing Alert storage.
f. Hostile and high-speed report output.
g. Identity statistics output.
h. Replay output.
i. Short README containing setup and execution steps.
j. Presentation that includes understanding of the project, architecture, JDBC flow, code-base.

14.1 Definition of Done

The project is complete only when the application runs from MTMSApplication, database rows can be verified using SQL, all required reports work, replay works, and the developer can explain why Service and Repository responsibilities are separated.

15. How is this requirement document connected to the Larger MTDL-SAD?

MTMS is not the final MTDL-SAD system. It is a foundation project that keeps the major backend flow but removes difficult technologies. Direct Track creation later becomes simulator output and network ingest. PostgreSQL remains the historical store. Replay, validation, filtering and alert ideas expand into larger services.

MTMS concept	Future larger-system concept
TrackSimulator creates objects directly	TDL simulator sends messages to an ingest layer.
ValidationService	Ingest validation and domain mapping.
TrackRepository with PostgreSQL	Historical Track persistence.
Simple reports	Filtering and operational APIs.
ReplayService	Time-window replay with selectable speed.
AlertService	Operational alert generation and acknowledgement.
Simple package structure	Spring Boot modules and service architecture.

16. Domain Primer

16.1 Real Object, Observation and Track

Term	What exists in the real world	What the software stores
Real object	A person, vehicle, aircraft, ship, drone or another moving item.	The software does not store the physical object itself.
Observation	A report saying that an object was seen at a place and time.	A set of measured or reported values.
Track	A continuing digital representation used to follow the object over time.	Track number, position, speed, identity, status and timestamp.

A useful everyday comparison is a delivery application. The delivery vehicle is real. The moving dot shown in the delivery application is a software representation. The dot, its latest location and its movement history are similar to a Track. The dot is not the vehicle itself.

16.2 What can be treated as a Track?

A soldier can be represented by a Track only when the system receives enough information to follow that soldier as a separate detected entity. In another situation, the system may represent a squad, team, vehicle or unit as one Track. The project does not decide which real-world level is used. MTMS simply demonstrates how one **identified (**valid**) moving entity** can be represented in software.

16.3 What Is Tactical Data?

For this simulation project, tactical data simply means time-sensitive information about objects and their situation, such as position, identity, speed and status. The developers only need to understand that this information (data) may change frequently and must be shared consistently.

16.4 What Is a Tactical Data Link?

A data link is a communication path that allows systems to exchange structured digital information. A tactical data link uses defined message formats so different participating systems can understand the same information. Official public material emphasizes standardization and interoperability, meaning systems must use compatible message rules.

For a simple analogy, imagine a messenger group used by aircraft, ships, ground stations and control centers. Participants send digital reports rather than informal chat.

16.5 What Does Multi-TDL Mean?

TDL means Tactical Data Link. Multi-TDL means information may come from more than one type of data link. In the full concept, the system must bring information from different links into one coherent view. MTMS does not connect to any real link. The simulator replaces all external links for learning purposes.

16.6 What Does Situational Awareness Mean?

Situational awareness means understanding what is happening around the operator: what objects exist, where they are, how they are moving, what their identity is and whether anything needs attention. In MTMS, the console reports are a very small substitute for a future visual display.

16.7 What Is a Situational Awareness Display?

It is a screen that presents the current picture to an operator. The full project may later show a map, symbols, tables and alerts. MTMS has no screen, so it prepares the backend information that such a display would eventually consume.

16.8 Business Story-line : Summarized version

This, our simulator creates artificial Track objects. Each Track contains information about one moving entity. The ValidationService checks whether the data makes sense (whether the identified Track object is valid or not). The TrackService decides what happens next. Valid data is saved in PostgreSQL through TrackRepository. Invalid data is not saved as a Track. Instead, AlertService creates an Alert and AlertRepository saves it. Other services later search, report and replay the saved history.